Taller práctico: Evolución de un sistema de alquiler de vehículos
Asignatura: Programación Orientada a Objetos
Duración sugerida: 2 horas
Lenguaje: C#

Contexto

Una empresa de movilidad urbana utiliza una aplicación para registrar vehículos disponibles para alquiler, calcular el valor de sus servicios y gestionar la recarga de aquellos que funcionan con batería

Un desarrollador construyó una primera versión del sistema. El proyecto compila y ejecuta una demostración básica, pero la empresa ha identificado situaciones que el diseño actual no resuelve correctamente. Tu equipo ha sido contratado para revisar el código existente y adaptarlo sin reconstruir toda la aplicación.

La empresa trabaja actualmente con bicicletas convencionales, patinetas eléctricas y automóviles eléctricos. Cada vehículo tiene un código que lo identifica. Las bicicletas cuestan $5.000 por hora. Las patinetas eléctricas cuestan $8.000 por hora, además de un cargo fijo de activación de $2.000 por alquiler. Los automóviles eléctricos cuestan $18.000 por hora y ofrecen un descuento del 10 % sobre el valor del alquiler cuando se contratan por cinco horas o más.

La empresa también ha aclarado que solo pueden registrarse vehículos de una categoría concreta. No tiene sentido alquilar algo identificado únicamente como vehículo sin especificar su modalidad. Aun así, desea conservar la información y las operaciones que comparten los diferentes tipos de vehículos.

En cuanto a la recarga, tanto las patinetas como los automóviles eléctricos cuentan con una batería cuyo nivel se expresa como porcentaje. La estación instalada hasta ahora únicamente admite patinetas; debe poder atender también automóviles y futuros vehículos recargables sin necesidad de añadir un método especial para cada clase. Una recarga solo es válida cuando la cantidad solicitada es positiva y el nivel final no supera el 100 %. Las operaciones inválidas deben rechazarse sin modificar el nivel almacenado.

Los encargados han pedido, además, que los cálculos de alquiler rechacen duraciones menores o iguales a cero y que el sistema pueda procesar una colección con distintos vehículos sin preguntar de qué tipo es cada uno para calcular su costo.

La empresa necesita administrar los vehículos que conforman su flota. Para ello, el sistema deberá permitir incorporar nuevos vehículos, retirar aquellos que ya no se encuentren disponibles y consultar la información de toda la flota.

Cada vehículo deberá identificarse mediante un código único, por lo que no se permitirá registrar dos vehículos con el mismo código. Asimismo, no será posible retirar un vehículo que no se encuentre registrado.

Al consultar la flota, el sistema deberá mostrar la información correspondiente a cada vehículo, incluyendo sus características particulares según la modalidad a la que pertenece.

La empresa desea que estas operaciones se realicen de manera centralizada, evitando que otros componentes del sistema puedan modificar directamente la colección de vehículos.

Trabajo solicitado

Revisa el proyecto entregado e identifica los problemas de diseño y funcionamiento que impiden cumplir las nuevas reglas. Modifica únicamente lo necesario para obtener una solución coherente, reutilizable y fácil de ampliar. Puedes ajustar las clases y los contratos existentes cuando encuentres responsabilidades que no correspondan a su propósito; conserva la funcionalidad correcta y evita duplicar código innecesariamente.

Al finalizar, la demostración en Program.cs debe permitir comprobar el valor de los alquileres de los tres tipos de vehículos, la recarga de una patineta y un automóvil mediante la misma operación de la estación, y el rechazo de operaciones inválidas sin dejar el sistema en un estado inconsistente.

Comprobaciones mínimas

Situación	**Resultado esperado**
Bicicleta por 3 horas	$15.000
Patineta por 3 horas	$26.000
Automóvil por 3 horas	$54.000
Automóvil por 5 horas	$81.000
Recargar patineta de 40 % con 30 puntos	70 %
Recargar automóvil de 60 % con 25 puntos	85 %
Intentar recargar patineta de 40 % con 70 puntos	Operación rechazada; conserva 40 %
Solicitar alquiler por 0 horas	Operación rechazada

Entrega

1. Proyecto C# modificado y ejecutable.

2. Realizar un párrafo del diagnostico encontrado.

3. Diagrama de clases UML de la solución Corregida.

4. Demostración de los casos anteriores desde Program.cs (o una prueba equivalente que pueda ejecutarse).

No existe una única implementación válida, pero la solución debe preservar la colaboración entre objetos y facilitar la incorporación de nuevos tipos de vehículos.

Ítem	Valor
Polimorfismo	4
Abstracción	3
Encapsulamiento	3
Interfaces	3
Herencia y reutilización	2
Funcionamiento (casos de prueba)	3
Diagnóstico	2
Total	**20**